

closure_evidence_20260922

BOB-224 closure evidence — 2026-09-22/23

Root cause (captured evidence — the initial hypothesis was tested and REFUTED, then the real cause found)

The investigation's first hypothesis (a background watchdog subshell inheriting a command-substitution's pipe write-end) was tested via process-tree/fd inspection and did NOT hold — no such watchdog exists in `scripts/compute-badges.sh`, and an isolated minimal reproducer of just `count_frontend_tests()` completed in ~13s with no hang.

The real root cause: `scripts/compute-badges.sh`'s `count_frontend_tests()` ran `timeout 120 "${vitest_bin}" list --run` with `NO -k/--kill-after` escalation. `timeout N <cmd>` with no `-k` sends only `SIGTERM` at the deadline and then `WAITS` for the child to exit — it does not forcibly bound execution. `vitest`'s own CLI installs a `process.once("SIGTERM", ...)` handler (Node's cooperative signal handling — the JS callback only runs once the event loop is free), and `vitest list --run` spends its opening phase in synchronous, CPU-heavy work (esbuild-transforming every `*.spec.ts` file). Under real host CPU/memory pressure (this session's swap was observed completely full, 8.0Gi/8.0Gi, during the investigation), that synchronous phase can starve the event loop well past the nominal 120s budget — so the “timeout” never actually fires, and the whole chain blocks until some OUTER wrapper (e.g. a 300s test-runner timeout) eventually `SIGKILLs` the entire tree. That matches “genuinely hangs... rc=124... after the full 300s” exactly.

Isolated, minimal, decisive reproduction: a Node script installing a SIGTERM handler then busy-looping synchronously for 8s, run under `timeout 3 node script.js (no -k)`, ran the FULL 8-second loop — the 3-second budget was completely unenforced. Confirmed against `timeout --help`'s own documented behaviour and against vitest's actual bundled CLI source (`frontend/node_modules/vitest/dist/chunks/cli-api.*.js`, which installs exactly this handler shape).

Fix

`scripts/compute-badges.sh` (the vitest invocation in `count_frontend_tests()`): `timeout 120 ... → timeout -k 10 120 ....` SIGKILL cannot be caught, blocked, or delayed by JS/event-loop state — a genuine hard bound (~130s worst case) instead of a cooperative one. Documented in-source with the full root-cause chain. Budget (120s) unchanged — this is not a “raise the timeout” workaround, and the suite is not silenced/skipped.

RED test (new): `tests/unit/test_compute_badges_frontend_timeout_bound.sh`

Per the item's own framing (“the failure IS the duration”), asserts TIMING, not function output: (1) reproduces the vulnerability class on this host/ toolchain — bare `timeout 2s (no -k)` lets a SIGTERM-catching busy loop overrun its budget; (2) the fixed shape (`timeout -k <grace> <budget>`) terminates the same stub within `budget+grace+margin`, with a timeout/kill exit status (124/137), not a clean exit; (3) a structural grep confirming the REAL `scripts/compute-badges.sh` carries the `-k` escalation on its actual vitest line (§11.4.201(11) — proves the real file is fixed, not just the abstract mechanism).

Verified genuinely RED pre-fix (via `git stash`, re-run, `git stash pop`): assertions 1/2 still passed (host/mechanism-only, fix-independent) but assertion 3 correctly FAILED — RESULT: 3 passed, 1 failed, exit 1.

Independently re-verified this session (coordinator)

```
$ bash -n scripts/compute-badges.sh && bash -n tests/unit/  
test_compute_badges_frontend_timeout_bound.sh  
(both clean)
```

```
$ timeout 60 bash tests/unit/  
test_compute_badges_frontend_timeout_bound.sh  
  PASS: vulnerability class reproduced: bare 'timeout 2s' (no -k)  
let a SIGTERM-catching busy loop run for 6061ms (budget was  
2000ms)  
  PASS: fix shape enforces a hard bound: 'timeout -k 2 2' killed  
the same busy-looping SIGTERM-catcher within 4028ms  
  PASS: timeout -k reported a timeout/kill exit status (rc=137)  
rather than a clean process exit  
  PASS: scripts/compute-badges.sh's vitest invocation carries a  
-k/--kill-after hard-bound escalation  
RESULT: 4 passed, 0 failed
```

```
$ time timeout 200 bash tests/unit/  
test_compute_badges_carrier_match.sh  
real 1m1.269s  
RESULT: 7 passed, 0 failed
```

The ORIGINALLY-HANGING suite (tests/unit/
test_compute_badges_carrier_match.sh, the item's own reported hang)
now completes in 61s with a clean exit=0 and 7/7 pass — previously it
hung to the full 300s budget with rc=124.

Determinism proof (subagent's own run, 3 consecutive clean runs each)

New regression test: 3/3 runs, rc=0, ~10s each, 4/4 assertions pass.
Original hanging suite: 3/3 runs, rc=0, ~61s each, 7/7 pass. (One earlier
set of 3 observed rc=124 at an investigator-imposed 60s bound —
explicitly NOT the residual hang; that was the investigation's own too-
tight loop timeout racing genuinely-heavy real work under the host's
actual swap-exhausted state at that moment, re-verified immediately
after at a 200s bound with a clean, consistent pass.)

git diff --stat

```
scripts/compute-badges.sh | 26 ++++
+++++
tests/unit/test_compute_badges_frontend_timeout_bound.sh | new
file, 7729 bytes
```